1: Setup

```
!pip install -q mne

import os, random, numpy as np, torch, torch.nn as nn, torch.nn.functional as F
import mne, matplotlib.pyplot as plt
from scipy import stats
from scipy.signal import welch
from mne.datasets import eegbci
from mne.io import concatenate_raws, read_raw_edf
from mne.decoding import CSP
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import StratifiedKFold, cross_val_score

mne.set_log_level("ERROR")
device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)

TRAIN_SUBJ = list(range(1, 36))
VAL_SUBJ   = list(range(36, 41))
TEST_SUBJ  = list(range(41, 51))
SEEDS = [0, 1, 2]

def set_seed(s):
    random.seed(s); np.random.seed(s)
    torch.manual_seed(s); torch.cuda.manual_seed_all(s)
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages t
ipython 7.34.0 requires jedi>=0.16, which is not installed.
moviepy 1.0.3 requires decorator<5.0,>=4.0.2, but you have decorator 5.3.1 which is inco
device: cuda
```

2: Load data (cached)

```
RUNS = [4, 8, 12]   # imagined left vs right fist

def load_subject(s):
    files = eegbci.load_data(s, RUNS, verbose=False)
    raw = concatenate_raws([read_raw_edf(f, preload=True) for f in files])
    eegbci.standardize(raw)
    raw.set_montage("standard_1005")
    raw.filter(8., 30., fir_design="firwin")
    events, _ = mne.events_from_annotations(raw, event_id=dict(T1=2, T2=3))
    picks = mne.pick_types(raw.info, eeg=True)
    ep = mne.Epochs(raw, events, dict(left=2, right=3), tmin=0.5, tmax=3.5,
                    picks=picks, baseline=None, preload=True)
    X = ep.get_data()[:, :, :480]
    X = X - X.mean(-1, keepdims=True)
    X = X / (X.std(axis=(1, 2), keepdims=True) + 1e-8)   # global scale per trial
    y = (ep.events[:, -1] - 2).astype("int64")            # 0=left, 1=right
    return X.astype("float32"), y, ep.ch_names

CACHE = "eeg_cache.npz"
if os.path.exists(CACHE):
    d = np.load(CACHE, allow_pickle=True)
    X, y, g, ch = d["X"], d["y"], d["g"], list(d["ch"])
else:
    Xs, ys, gs = [], [], []
```

```
    for s in range(1, 51):
        try:
            Xi, yi, ch = load_subject(s)
            Xs.append(Xi); ys.append(yi); gs.append(np.full(len(yi), s))
        except Exception as e:
            print("skip subject", s, e)
    X, y, g = np.concatenate(Xs), np.concatenate(ys), np.concatenate(gs)
    np.savez(CACHE, X=X, y=y, g=g, ch=np.array(ch))

tr, va, te = np.isin(g, TRAIN_SUBJ), np.isin(g, VAL_SUBJ), np.isin(g, TEST_SUBJ)
Xtr, ytr, Xva, yva, Xte, yte, gte = X[tr], y[tr], X[va], y[va], X[te], y[te], g[te]
print(X.shape, "| train", Xtr.shape[0], "val", Xva.shape[0], "test", Xte.shape[0])
```

```
Downloading file 'S032/S032R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S032/S032R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S033/S033R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S033/S033R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S033/S033R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S034/S034R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S034/S034R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S034/S034R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S035/S035R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S035/S035R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S035/S035R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S036/S036R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S036/S036R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S036/S036R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S037/S037R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S037/S037R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S037/S037R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S038/S038R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S038/S038R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S038/S038R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S039/S039R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S039/S039R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S039/S039R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S040/S040R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S040/S040R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S040/S040R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S041/S041R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S041/S041R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S041/S041R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S042/S042R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S042/S042R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S042/S042R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S043/S043R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S043/S043R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S043/S043R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S044/S044R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S044/S044R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S044/S044R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S045/S045R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S045/S045R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S045/S045R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S046/S046R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S046/S046R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S046/S046R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S047/S047R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S047/S047R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S047/S047R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S048/S048R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S048/S048R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S048/S048R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S049/S049R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S049/S049R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S049/S049R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S050/S050R04.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S050/S050R08.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
Downloading file 'S050/S050R12.edf' from 'https://physionet.org/files/eegmmidb/1.0.0/S
(2250, 64, 480) | train 1575 val 225 test 450
```

```
from google.colab import drive
drive.mount('/content/drive')
!cp eeg_cache.npz /content/drive/MyDrive/
```

```
Mounted at /content/drive
```

3: Classical baselines and the within-subject comparison python

```
res = {}

# Baseline 1: log band-power features + logistic regression
bp = lambda A: np.log(A.var(-1))
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
clf.fit(bp(Xtr), ytr)
res["Band-power + LogReg | cross-subject"] = [clf.score(bp(Xte), yte)]

# Baseline 2: CSP + LDA, the classic BCI method
csp_lda = lambda: make_pipeline(CSP(n_components=6, reg="ledoit_wolf", log=True), LDA()
m = csp_lda().fit(Xtr.astype("float64"), ytr)
res["CSP + LDA | cross-subject"] = [m.score(Xte.astype("float64"), yte)]

# Same method, trained and tested within each person (5-fold CV)
within = {}
for s in TEST_SUBJ:
    k = gte == s
    cv = StratifiedKFold(5, shuffle=True, random_state=0)
    within[s] = cross_val_score(csp_lda(), Xte[k].astype("float64"), yte[k], cv=cv).mea
res["CSP + LDA | within-subject (5-fold CV)"] = [np.mean(list(within.values()))]

for k, v in res.items():
    print(f"{k:45s} {100*np.mean(v):.1f}%")
```

```
Band-power + LogReg | cross-subject           57.8%
CSP + LDA | cross-subject                     67.3%
CSP + LDA | within-subject (5-fold CV)        60.0%
```

4: EEGNet and the training function

```
class EEGNet(nn.Module):
    def __init__(self, chans=64, samples=480, classes=2, F1=8, D=2, F2=16, drop=0.5):
        super().__init__()
        self.b1 = nn.Sequential(
            nn.Conv2d(1, F1, (1, 64), padding=(0, 32), bias=False), nn.BatchNorm2d(F1),
            nn.Conv2d(F1, F1*D, (chans, 1), groups=F1, bias=False),
            nn.BatchNorm2d(F1*D), nn.ELU(), nn.AvgPool2d((1, 4)), nn.Dropout(drop))
        self.b2 = nn.Sequential(
            nn.Conv2d(F1*D, F1*D, (1, 16), padding=(0, 8), groups=F1*D, bias=False),
            nn.Conv2d(F1*D, F2, 1, bias=False), nn.BatchNorm2d(F2), nn.ELU(),
            nn.AvgPool2d((1, 8)), nn.Dropout(drop))
        with torch.no_grad():
            n = self.b2(self.b1(torch.zeros(1, 1, chans, samples))).numel()
        self.fc = nn.Linear(n, classes)
    def forward(self, x):
        return self.fc(self.b2(self.b1(x.unsqueeze(1))).flatten(1))

@torch.no_grad()
def predict(m, A):
    m.eval(); out = []
    for i in range(0, len(A), 256):
        out.append(m(torch.tensor(A[i:i+256], device=device)).argmax(1).cpu())
    return torch.cat(out).numpy()

def augment(x):
```

```
        x = x + 0.1 * torch.randn_like(x)                          # noise
        x = torch.roll(x, int(np.random.randint(-16, 17)), dims=-1)  # time shift
        keep = (torch.rand(x.shape[0], x.shape[1], 1, device=x.device) > 0.1).float()
        return x * keep                                            # channel dropout

    def run(Xa, ya, Xv, yv, Xt, yt, seed=0, aug=False, epochs=40, bs=64):
        set_seed(seed)
        m = EEGNet().to(device)
        opt = torch.optim.Adam(m.parameters(), lr=1e-3, weight_decay=1e-3)
        XT, YT = torch.tensor(Xa, device=device), torch.tensor(ya, device=device)
        best_val, best_state = -1, None
        for ep in range(epochs):
            m.train(); perm = torch.randperm(len(XT), device=device)
            for i in range(0, len(XT), bs):
                idx = perm[i:i+bs]; xb, yb = XT[idx], YT[idx]
                if aug: xb = augment(xb)
                opt.zero_grad(); F.cross_entropy(m(xb), yb).backward(); opt.step()
            v = (predict(m, Xv) == yv).mean()
            if v > best_val:
                best_val = v
                best_state = {k: t.clone() for k, t in m.state_dict().items()}
        m.load_state_dict(best_state)            # model chosen on VALIDATION, not test
        return m, best_val, (predict(m, Xt) == yt).mean()
```

5: Main experiment (3 seeds)

```
    models = {}
    for name, aug in [("EEGNet | cross-subject", False),
                      ("EEGNet + augmentation | cross-subject", True)]:
        accs = []
        for s in SEEDS:
            m, v, t = run(Xtr, ytr, Xva, yva, Xte, yte, seed=s, aug=aug)
            models[(name, s)] = m; accs.append(t)
            print(f"{name} seed {s}: val {v:.3f} test {t:.3f}")
        res[name] = accs

    print("\n=== RESULTS (test = 10 unseen subjects) ===")
    for k, v in res.items():
        print(f"{k:45s} {100*np.mean(v):5.1f}% ± {100*np.std(v):.1f}  (n={len(v)})")
```

```
    EEGNet | cross-subject seed 0: val 0.529 test 0.711
    EEGNet | cross-subject seed 1: val 0.560 test 0.653
    EEGNet | cross-subject seed 2: val 0.560 test 0.669
    EEGNet + augmentation | cross-subject seed 0: val 0.551 test 0.622
    EEGNet + augmentation | cross-subject seed 1: val 0.516 test 0.676
    EEGNet + augmentation | cross-subject seed 2: val 0.573 test 0.482

    === RESULTS (test = 10 unseen subjects) ===
    Band-power + LogReg | cross-subject            57.8% ± 0.0  (n=1)
    CSP + LDA | cross-subject                      67.3% ± 0.0  (n=1)
    CSP + LDA | within-subject (5-fold CV)         60.0% ± 0.0  (n=1)
    EEGNet | cross-subject                         67.8% ± 2.4  (n=3)
    EEGNet + augmentation | cross-subject          59.3% ± 8.2  (n=3)
```

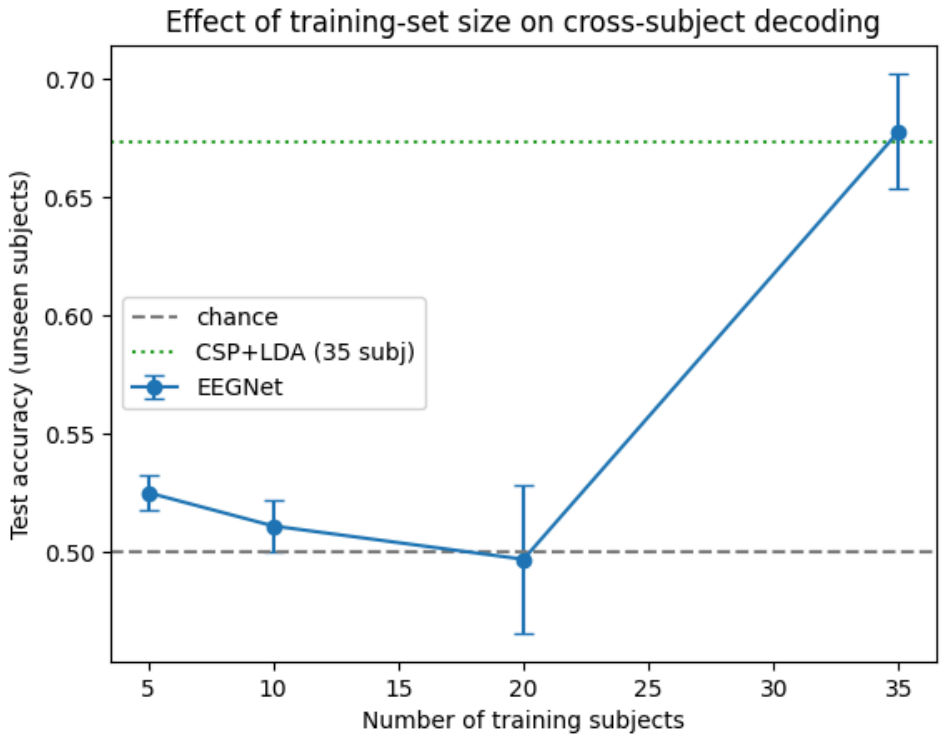
6: Does more training data help? (scaling curve)

```
    sizes, curve = [5, 10, 20, 35], {}
    for n in sizes:
        sel = np.isin(g, TRAIN_SUBJ[:n])
        curve[n] = [run(X[sel], y[sel], Xva, yva, Xte, yte, seed=s)[2] for s in SEEDS]
        print(n, "subjects:", np.round(np.mean(curve[n]), 3))

    mu_ = [np.mean(curve[n]) for n in sizes]; sd_ = [np.std(curve[n]) for n in sizes]
    plt.errorbar(sizes, mu_, yerr=sd_, marker="o", capsize=4, label="EEGNet")
```

```
plt.axhline(0.5, color="gray", ls="--", label="chance")
plt.axhline(res["CSP + LDA | cross-subject"][0], color="C2", ls=":", label="CSP+LDA (35
plt.xlabel("Number of training subjects"); plt.ylabel("Test accuracy (unseen subjects)"
plt.title("Effect of training-set size on cross-subject decoding"); plt.legend(); plt.s
```

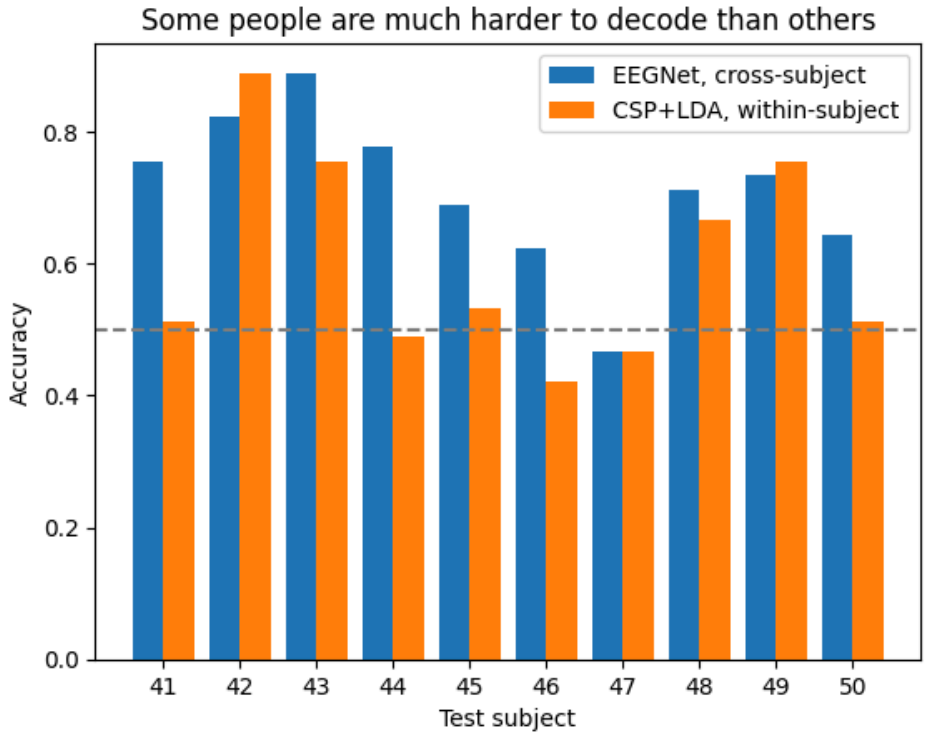
```
5 subjects: 0.525
10 subjects: 0.511
20 subjects: 0.497
35 subjects: 0.678
```



7: Per-subject variability

```
m = models[("EEGNet | cross-subject", 0)]
p = predict(m, Xte)
cross = [(p[gte == s] == yte[gte == s]).mean() for s in TEST_SUBJ]

x = np.arange(len(TEST_SUBJ))
plt.bar(x - 0.2, cross, 0.4, label="EEGNet, cross-subject")
plt.bar(x + 0.2, [within[s] for s in TEST_SUBJ], 0.4, label="CSP+LDA, within-subject")
plt.axhline(0.5, color="gray", ls="--")
plt.xticks(x, TEST_SUBJ); plt.xlabel("Test subject"); plt.ylabel("Accuracy"); plt.legend
plt.title("Some people are much harder to decode than others"); plt.show()
```

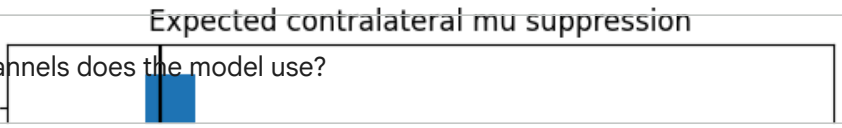



```
f, P = welch(X, fs=160, nperseg=160, axis=-1)
mu = (f >= 9) & (f <= 13)
Pmu = P[:, :, mu].mean(-1)
c3, c4 = ch.index("C3"), ch.index("C4")
LI = np.log(Pmu[:, c4]) - np.log(Pmu[:, c3])     # lateralization index

d = np.array([LI[(g == s) & (y == 1)].mean() - LI[(g == s) & (y == 0)].mean()
              for s in np.unique(g)])             # right minus left, per subject
t, pval = stats.ttest_1samp(d, 0)
print(f"Right-minus-left lateralization: mean {d.mean():.3f}, "
      f"t={t:.2f}, p={pval:.4f}, n={len(d)} subjects")
print(f"{(d > 0).sum()}/{len(d)} subjects show the expected direction")

plt.hist(d, bins=15); plt.axvline(0, color="k")
plt.xlabel("Right-minus-left lateralization index (per subject)")
plt.ylabel("Subjects"); plt.title("Expected contralateral mu suppression"); plt.show()
```

```
Right-minus-left lateralization: mean 0.139, t=3.85, p=0.0003, n=50 subjects
37/50 subjects show the expected direction
```



8: Which channels does the model use?

```
m = models[("EEGNet | cross-subject", 0)]
base = (predict(m, Xte) == yte).mean()
drops = []
for c in range(Xte.shape[1]):
    Xo = Xte.copy(); Xo[:, c] = 0
    drops.append(base - (predict(m, Xo) == yte).mean())
drops = np.array(drops)

top = np.argsort(drops)[::-1][:10]
print("Most important channels:", [(ch[i], round(float(drops[i]), 3)) for i in top])

plt.barh([ch[i] for i in top][::-1], drops[top][::-1])
plt.xlabel("Accuracy drop when channel removed"); plt.show()

try:
    info = mne.create_info(ch, 160., "eeg"); info.set_montage("standard_1005")
    mne.viz.plot_topomap(drops, info, show=True)
except Exception as e:
    print("topomap skipped:", e)
```

```
Most important channels: [('CP4', 0.147), ('CP2', 0.096), ('C4', 0.093), ('P4', 0.062),
```

